

# Trabalho Prático 1 - Álgebra A

Luiz Felipe Gondim Araujo

Universidade Federal de Minas Gerais (UFMG)  
Belo Horizonte - MG - Brasil  
Matrículas: 2023028188

## Introdução

Este relatório apresenta o desenvolvimento e a fundamentação teórica das implementações realizadas para o Trabalho Final da disciplina de Álgebra A. O objetivo principal do projeto é demonstrar a compreensão prática e matemática de conceitos da teoria aritmética dos números e de criptografia por meio da construção autônoma de algoritmos clássicos. O documento a seguir detalha a base teórica que garante a corretude das operações, as principais decisões de implementação adotadas no código-fonte e a verificação estruturada dos algoritmos por meio dos casos de teste exigidos e cenários adicionais, evidenciando a consolidação do conhecimento adquirido ao longo do curso.

## 1 Fundamentação Teórica

### 1.1 Teste de Fermat e Números de Carmichael

O Teste Probabilístico de Fermat fundamenta-se no Pequeno Teorema de Fermat, que estabelece que, se  $p$  é um número primo e  $a$  é um inteiro tal que  $\gcd(a, p) = 1$ , então:

$$a^{p-1} \equiv 1 \pmod{p}$$

O algoritmo utiliza a contrapositiva desse teorema, ou seja, dado um inteiro  $n > 1$ , escolhe-se uma base  $a$  aleatória com  $1 < a < n$ . Se  $\gcd(a, n) > 1$ , conclui-se que  $n$  é composto. Contudo, a recíproca do teorema não é verdadeira, existem números compostos, denominados Números de Carmichael, que satisfazem  $a^{n-1} \equiv 1 \pmod{n}$  para toda base  $a$  coprima com  $n$ . Devido a essa propriedade, os números de Carmichael (como 561, 1105 e 1729) atuam como falsos positivos universais neste teste, exigindo métodos mais rigorosos como o teste de Miller-Rabin para garantir a primalidade criptográfica.

### 1.2 Teste de Miller-Rabin

Para contornar as falhas do Teste de Fermat, utiliza-se o Teste de Miller-Rabin. Dado um número ímpar  $n$ , podemos fatorar as potências de 2 de  $n - 1$ , reescrevendo-o na forma  $n - 1 = 2^s \cdot d$ , onde  $d$  é ímpar. O teste baseia-se em um lema fundamental da aritmética modular, isso quer dizer que se  $p$  é primo, as únicas raízes da congruência  $x^2 \equiv 1 \pmod{p}$  são  $x \equiv 1 \pmod{p}$  e  $x \equiv -1 \pmod{p}$ .

Portanto, para uma base aleatória  $a$ , consideramos a sequência de quadrados:

$$a^d, a^{2d}, a^{4d}, \dots, a^{2^{s-1}d} \pmod{n}$$

Se  $n$  for primo, o primeiro valor da sequência deve ser 1 ou, caso contrário, o valor 1 só pode aparecer se for imediatamente precedido por  $-1$  (que equivale a  $n - 1 \pmod{n}$ ). Se o algoritmo encontra um valor 1 precedido por qualquer número que não seja  $\pm 1$ , descobre-se um divisor não trivial de zero, provando que  $n$  é composto.

### 1.3 Criptossistema RSA

A segurança do RSA baseia-se na dificuldade computacional da fatoração de inteiros grandes. A geração de chaves exige dois primos grandes  $p$  e  $q$ , calculando-se o módulo  $n = pq$  e a função totiente de Euler  $\phi(n) = (p - 1)(q - 1)$ . A chave pública  $e$  é escolhida de modo que  $\gcd(e, \phi(n)) = 1$ , e a chave privada  $d$  é o inverso modular de  $e$ , satisfazendo  $ed \equiv 1 \pmod{\phi(n)}$ .

A corretude das funções de cifração ( $c = m^e \pmod{n}$ ) e decifração ( $m = c^d \pmod{n}$ ) é garantida pelo Teorema de Euler. Como  $ed = 1 + k\phi(n)$  para algum inteiro  $k$ , temos:

$$m^{ed} = m^{1+k\phi(n)} = m \cdot (m^{\phi(n)})^k \equiv m \cdot 1^k \equiv m \pmod{n}$$

Isso demonstra que  $Dec(Enc(m)) = m$ , recuperando a mensagem original.

## 2 Detalhes de Implementação

O projeto foi desenvolvido em C++11, priorizando a ausência de bibliotecas criptográficas prontas e implementando o controle do uso de memória para lidar com números da ordem de 64 bits.

### 2.1 Estruturas de Dados e Tipagem

A exigência de primos de 32 bits ( $p$  e  $q$ ) implica que o módulo  $n = pq$  alcançará até 64 bits. Para comportar esses valores e evitar *overflow* de memória, os tipos primitivos adotados em toda a aplicação foram o `unsigned long long`, garantindo 64 bits sem sinal, e o tipo nativo do GCC `unsigned __int128`.

### 2.2 Funções do Tópico 5: Fermat e Carmichael

O módulo de testes probabilísticos e análise de Carmichael é composto pelas seguintes funções principais:

- **gcd:** Implementa o algoritmo de Euclides tradicional para o cálculo do Máximo Divisor Comum, garantindo que não sejam usadas implementações nativas da linguagem.
- **powMod:** Substitui a função nativa de potência. Sabendo disso, ela utiliza a exponenciação binária, realizando conversão temporária para `unsigned __int128` durante as multiplicações para prevenir o *overflow* antes de aplicar a redução modular.
- **fermatTest:** Executa o teste probabilístico padrão sorteando  $k$  bases aleatórias no intervalo apropriado, atestando primalidade ou detectando composição.
- **verifyAllCoprimeBases:** Função analítica que itera sobre todas as bases  $a$  no intervalo  $[2, n - 1]$ . Utilizando a função `gcd`, isola apenas as bases coprimas a  $n$  e verifica se a congruência de Fermat se mantém, sendo utilizada para comprovar matematicamente a propriedade dos números de Carmichael.
- **runCarmichaelExamples:** Automatiza a demonstração dos casos exigidos (561, 1105 e 1729), explicitando suas fatorações e atestando que atuam como falsos positivos em um teste probabilístico simples.

## 2.3 Funções do Tópico 9: Miller-Rabin e RSA

O módulo de criptografia e primalidade forte engloba as rotinas de geração e manipulação de chaves:

- **extendedEuclidean e modularInverse:** A versão estendida encontra os coeficientes da Identidade de Bézout de forma recursiva. Estes coeficientes são repassados ao **modularInverse** para determinar a chave privada  $d$  a partir do expoente público  $e$  e do valor de  $\phi(n)$ .
- **millerRabinTest:** Realiza a fatoração das potências de 2 de  $n - 1$  e executa as validações das raízes da unidade módulo  $n$  por  $k$  rodadas. Diferente de Fermat, este teste detecta corretamente números de Carmichael, providenciando a primalidade necessária ao RSA.
- **generatePrime (Integração de Módulos):** Encapsula a geração criptográfica das chaves e estabelece a integração entre os tópicos. Utiliza a biblioteca `<random>` para gerar candidatos ímpares de 32 bits e cada candidato atua em uma arquitetura de funil, isso quer dizer que ele é imediatamente passado para o **fermatTest(n, 5)** atuando como um pré-filtro computacionalmente barato. Apenas se o candidato passar por esta etapa ele é submetido ao **millerRabinTest(n, 10)** para a validação criptográfica definitiva.
- **runRSACryptosystem:** Além de gerar as chaves e garantir via **while** que  $e$  seja coprimo a  $\phi(n)$  iterativamente, automatiza a verificação de corretude estipulada pelo problema. O laço força o teste de, no mínimo, 20 mensagens aleatórias, verificando integralmente se  $Dec(Enc(m)) = m$  e se as assinaturas digitais geradas são válidas.

## 2.4 Fluxo Principal e Robustez do Sistema

A interação com o usuário é gerenciada pelo arquivo `main.cpp`, que contém a `displayMenu` e o escopo da função `main` e interage com os módulos de criptografia. O sistema busca também ser robusto para evitar falhas de execução decorrentes de entradas malformadas do usuário. Então o iterador principal da aplicação implementa tratamento direto no fluxo de leitura, caso o usuário insira caracteres textuais ou não-numéricos quando um valor inteiro é esperado, o sistema ignora automaticamente. Essa blindagem previne que o programa entre em *loops* infinitos e bloqueia eventuais *segmentation faults*, assegurando a robustez desejada.

## 2.5 Dificuldades Encontradas

A principal dificuldade na implementação concentrou-se na aritmética computacional durante o cálculo de exponenciação modular. A operação iterativa `res = (res * base) % mod`, quando processando parâmetros criptográficos reais (onde `base` e `res` possuíam ambos perto de 64 bits de tamanho), gerava estouros de registrador durante a etapa de multiplicação, resultando em cálculos modulares falhos e, conseqüentemente, chaves privadas  $d$  corrompidas. O problema foi resolvido mapeando a multiplicação interna para 128 bits e aplicando o módulo antes do *downcast* novamente para 64 bits. Outra dificuldade foi assegurar que o expoente público clássico  $e = 65537$  fosse de fato coprimo a  $\phi(n)$ , resolvido com a inclusão de um laço reativo que incrementa  $e$  em 2 em casos de falha na validação do `gcd`.

## 2.6 Uso de Inteligencia Artificial

Assim como estabelecido, o uso de IA se restringiu a apenas sanar dúvidas conceituais do funcionamento do Miller-Rabin e da Criptografia RSA. Ademais, ainda foi usada para tentar resolver o problema do estouro de memória durante a etapa de multiplicação citada acima. O modelo utilizado foi o Claude Sonnet 4.6.

### 3 Exemplos e Resultados

#### 3.1 Testes do Tópico 5: Primalidade e Números de Carmichael

Os testes validaram a eficiência do Teste de Fermat e a identificação de falsos positivos quando os números de Carmichael foram utilizados.

| Número ( $n$ ) | Tipo Real  | Resultado ( $k = 10$ ) | Conclusão     |
|----------------|------------|------------------------|---------------|
| 104729         | Primo      | Provavelmente Primo    | Correto       |
| 341            | Composto   | Composto               | Correto       |
| 561            | Carmichael | Provavelmente Primo    | Erro Esperado |

Tabela 1: Resumo dos Testes de Primalidade

#### 3.2 Testes do Tópico 9: Criptossistema RSA

O sistema foi submetido a baterias de testes automáticos e manuais para garantir a robustez das operações de cifração e assinatura.

##### 3.2.1 Exemplo de Execução Automatizada de 20 testes (Resumida):

Teste 1

Mensagem original: 5391474601685057227

Cifrada: 2290523755841414199

Decifrada: 5391474601685057227

Recuperação: Sucesso

Teste 10

Mensagem original: 4851252149645429498

Cifrada: 5529265044463848243

Decifrada: 4851252149645429498

Recuperação: Sucesso

Teste 20

Mensagem original: 719260137038857179

Cifrada: 1786933590632331891

Decifrada: 719260137038857179

Recuperação: Sucesso

Resultado: Corretude verificada para 20 mensagens aleatórias.

##### 3.2.2 Exemplo de Execução manual com $m > n$ :

--- Gerando Chaves RSA ---

Primos Gerados:  $p = 3456088309$ ,  $q = 3298263541$

Parâmetros:  $n = 11399090064051042169$ ,  $e = 65537$ ,  $d = 5415772267788379793$

Escolha o modo de teste:

[1] Automático

[2] Manual

Opção: 2

Digite a mensagem (menor que 11399090064051042169): 9430580348503485098340580485  
Mensagem maior ou igual a n, ocorrerá perda de dados.

### 3.2.3 Exemplo de Execução manual para casos extremos:

Foram feitos testes para os casos onde a mensagem possui valor 0 ou 1.

--- Gerando Chaves RSA ---

Primos Gerados: p = 4140969643, q = 4110556427

Parâmetros: n = 17021689380045545561, e = 65537, d = 882610994002808565

Escolha o modo de teste:

[1] Automático

[2] Manual

Opção: 2

Digite a mensagem (menor que 17021689380045545561): 0

Cifrada: 0

Decifrada: 0

Recuperação: Sucesso

Testar outra mensagem? (s/n): s

Digite a mensagem (menor que 17021689380045545561): 1

Cifrada: 1

Decifrada: 1

Recuperação: Sucesso

## 4 Resultados e Discussões

### 4.1 Análise da Primalidade e Números de Carmichael

Os resultados corroboram a fundamentação teórica. O Teste de Fermat mostrou-se rápido para a maioria dos compostos, mas a falha sistemática nos números de Carmichael confirma que testes probabilísticos simples são insuficientes para fins criptográficos. Esta observação justifica a implementação da camada adicional de segurança via Teste de Miller-Rabin que valida a primalidade.

### 4.2 Avaliação da implementação RSA

A implementação do modo manual permitiu observar os seguintes comportamentos:

- **Mensagens aleatórias:** Foram geradas 20 mensagens aleatórias para testar a corretude do algoritmo de RSA e o teste funcionou para todos os casos.
- **Mensagens  $m \geq n$ :** Ao inserir valores superiores ao módulo, o sistema detectou a condição e emitiu um aviso de perda de dados. Isso ocorre porque o RSA opera sobre o grupo multiplicativo  $\mathbb{Z}_n^*$ , então valores fora do intervalo permitem apenas uma recuperação ambígua ( $m \pmod n$ ), confirmando a limitação do algoritmo RSA "puro" sem tratamento de entrada.

- **Mensagens  $m = 0$  e  $m = 1$ :** Os testes confirmaram a "identidade" dessas mensagens ( $0^e \equiv 0$  e  $1^e \equiv 1$ ). Embora matematicamente corretos, estes casos ilustram a ausência de ocultação de conteúdo.

## Conclusão

O desenvolvimento deste trabalho permitiu a aplicação prática dos fundamentos da teoria dos números, consolidando a compreensão sobre a primalidade e os mecanismos de segurança do sistema RSA. A implementação dos testes de Fermat e Miller-Rabin evidenciou a transição necessária entre modelos probabilísticos simples e algoritmos robustos capazes de identificar números de Carmichael, garantindo assim a integridade dos parâmetros criptográficos. Além disso, a gestão de dados de 64 bits e o tratamento rigoroso do fluxo de entrada demonstraram a importância de uma implementação defensiva para assegurar a correteza matemática em sistemas computacionais.

Por fim, o projeto confirmou que, embora o algoritmo RSA seja teoricamente sólido, sua implementação prática exige atenção a detalhes principalmente em relação a escolha de expoentes coprimos e o manejo de mensagens em casos de borda. A integração bem sucedida entre a geração de primos e a orquestração do RSA validou não apenas a funcionalidade do código, mas também a eficiência da arquitetura de funil adotada, que equilibra custo computacional e segurança. Com isso, o sucesso dos testes realizados confirmam que os objetivos do trabalho foram alcançados.

## Referências

- GEEKSFORGEES. **RSA Algorithm in Cryptography**. Disponível em: <https://www.geeksforgeeks.org/rsa-algorithm-cryptography/>. Acesso em: 5 jun. 2026.
- OPENSLL DOCUMENTATION. **Miller-Rabin Primality Test**. Disponível em: [https://www.openssl.org/docs/man3.0/man3/BN\\_is\\_prime\\_ex.html](https://www.openssl.org/docs/man3.0/man3/BN_is_prime_ex.html). Acesso em: 10 jun. 2026.
- WOLFRAM MATHWORLD. **Carmichael Number**. Disponível em: <https://mathworld.wolfram.com/CarmichaelNumber.html>. Acesso em: 9 jun. 2026.